

Team Member Full Name	NetID
Matthew Lee	mlee55
Sofia Cardona	scardon2
Martin Castellanos	mcastel5

Persistent Storage Design

We are using a SQLite database to persist the application's data. As shown in *Figure 1*, the database contains four main tables: *auth_user*, *player_profile*, *game*, and *guess*. These tables support user accounts, player information, gameplay history, and individual guesses.

The *auth_user* table stores login information such as username, email, and password. The *player_profile* table stores additional player-specific data, including the optional name field and *extra_plays_remaining*, which tracks how many purchased extra plays a user still has available. Each profile is linked to exactly one user.

The *game* table stores each gameplay session, including the player, selected language, target word, result, number of attempts, and creation time. The *guess* table stores the guesses made during each game, along with the feedback result for each guess. Together, these tables allow the application to manage player accounts, store game progress, support the dashboard, and handle purchased extra plays.

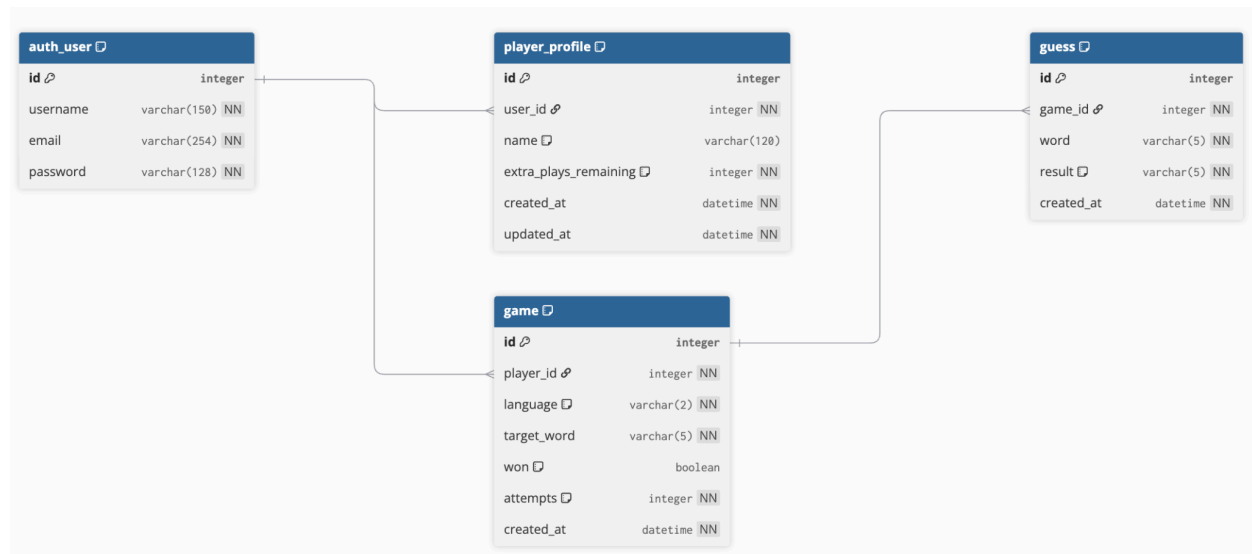


Figure 1. database schema

Demonstration of the Features

Feature 1: Sign up / Sign in

Feature 1.1: Create new User Profile

Subfeature 1.1 allows a user to create a new player profile in Worndly. On the profile creation page, the user enters an optional name and the required username, email, and password fields. When the form is submitted, the system validates the input and creates a new account.

After successful profile creation, the system stores the user's information in the database and redirects the user to their profile page, where the created profile information can be viewed. This feature establishes the initial account/profile creation flow for the application.

The figure consists of two screenshots of the Worndly application interface.

The top screenshot shows the "Create a new player profile" page. The header includes the "Worndly" logo and navigation links for "Profiles", "Create Profile", and "Admin". The page title is "SUBFEATURE 1.1". The main heading is "Create a new player profile", with a "View profiles" button to the right. The form contains four input fields: "Name:" (optional), "Username:", "Email:", and "Password:". Below the fields is a "Create profile" button. A note at the bottom states: "Name is optional. Username, email, and password are required by the Phase 1 specification."

The bottom screenshot shows the "Player Profile" page for a user named "Matthew". The header is the same as the top screenshot. The page title is "PLAYER PROFILE". The main heading is "Matthew", with a "Back to profiles" button to the right. Below the heading is a table displaying the user's profile information:

Name	Matthew
Username	Matthew
Email	mlee55@nd.edu
Created	April 8, 2026

Figure 2. Screenshot for Feature 1.1 showing creating player profile and created player profile

Feature 1.2: Log-in

Subfeature 1.2 allows a user to log into their existing Wornly account using their username and password. On the login page, the user enters their credentials into the required fields. Upon

submission, the system validates the input and authenticates the user against stored account information.

After a successful login, the system grants access to the user's account and redirects them to the main dashboard, where they can view their game history and navigate the application. This feature establishes secure access to existing user profiles and enables continued interaction with the platform's core functionality.

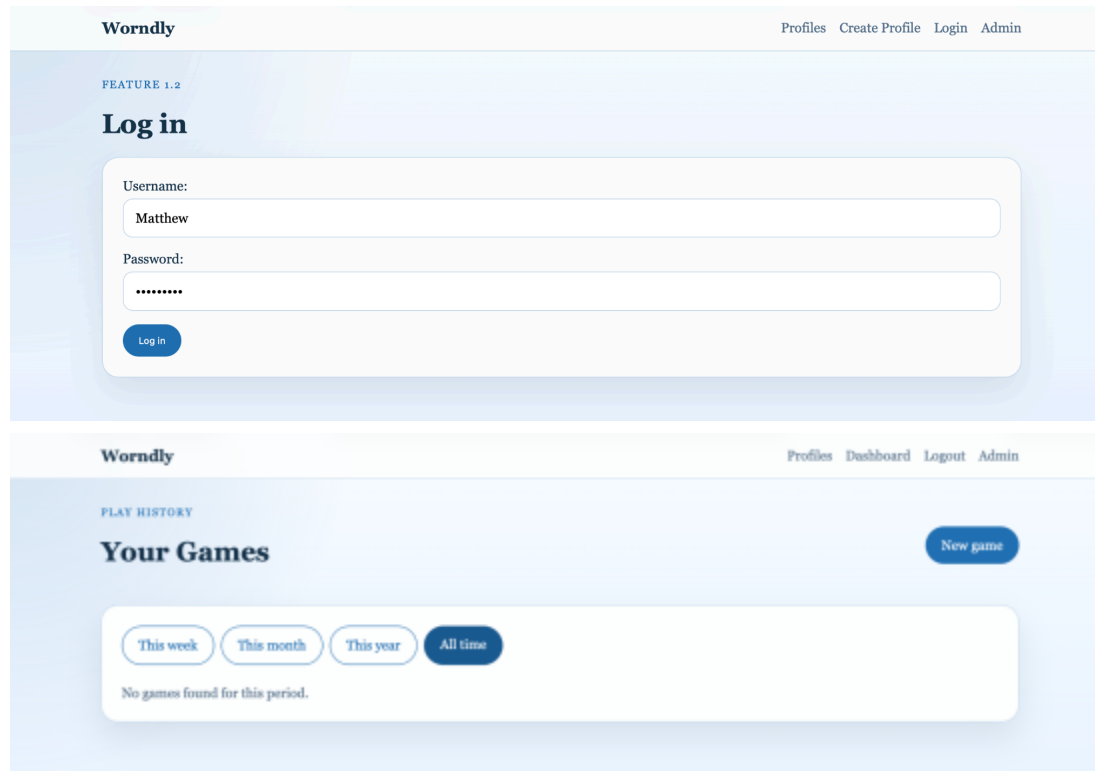


Figure 3. Screenshot for Feature 1.2 showing login screen and home dashboard

Feature 1.3: Log-out

Subfeature 1.3 allows a user to securely log out of their Worndly account. When the user selects the logout option from the navigation bar, the system terminates the active session and clears any stored authentication data associated with the user.

After logging out, the system redirects the user to the home page, where only non-authenticated features such as profile creation and login are accessible. This feature ensures account security by preventing unauthorized access and properly ending the user's session within the application.

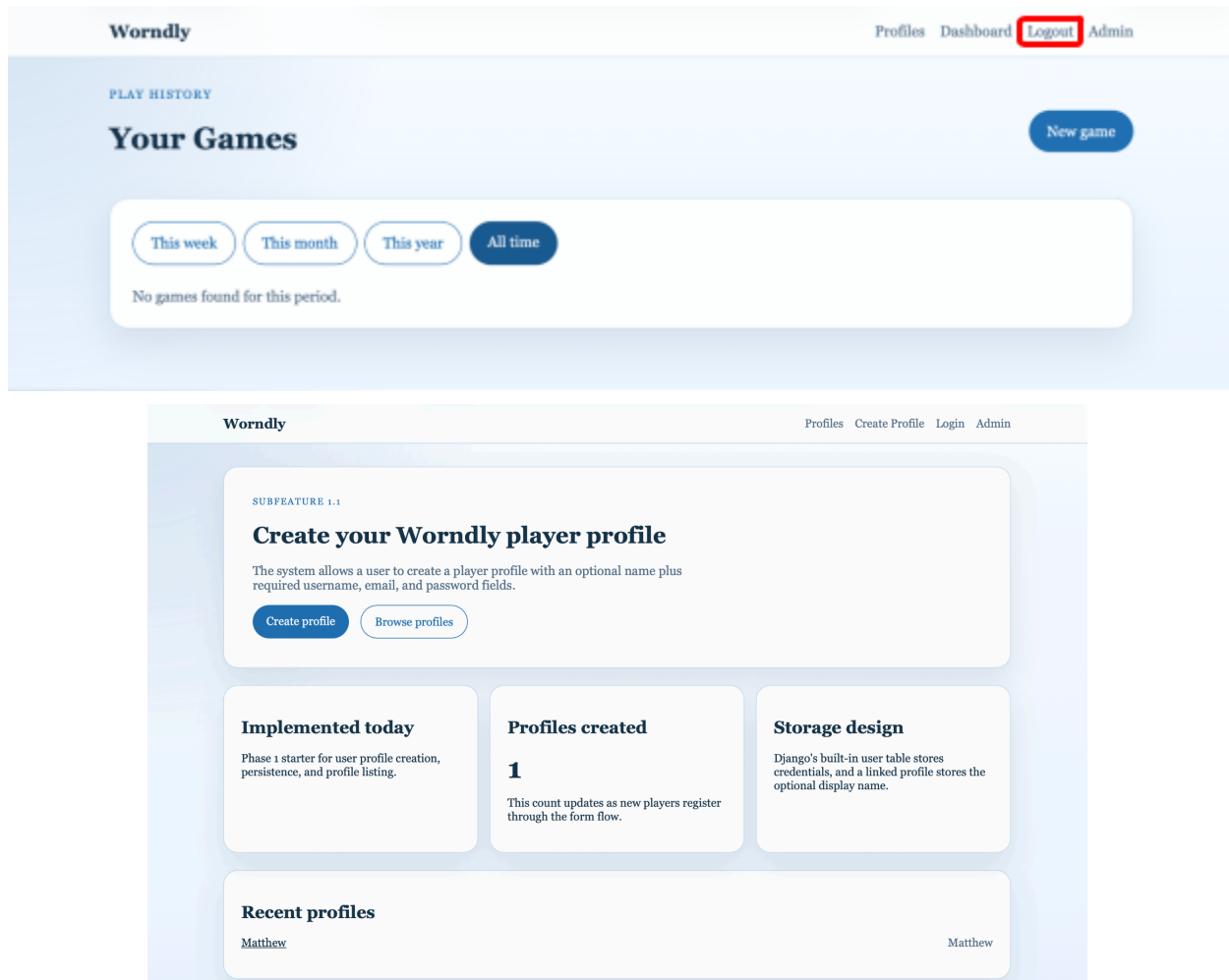


Figure 4. Screenshot for Feature 1.3 showing logout button (circled in red) and home page destination after clicking

Feature 2: Gameplay

This feature allows logged users to play WorNDly, which is a Wordle-like game. The user can select one of four languages (english, spanish, french, german, portuguese) and 'Start Game'. The users will be presented with blank tiles to fill in with five-letter words and have six attempts. In addition, the user is allowed three games per day. As the player inputs words, each letter is marked with a representative color: gray – letter not in word, yellow – letter in word but in wrong spot, green – correct letter in corresponding spot. If they input an invalid word, an error appears. Once the user guesses the correct word they are given the choice to play again; if they get it incorrect, the correct word is displayed and are given the choice to play again.

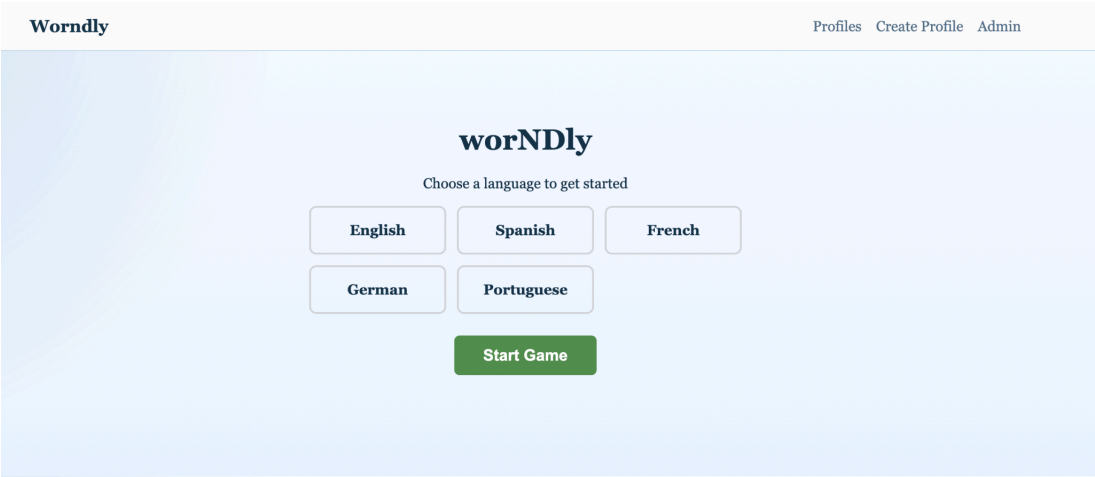


Figure 5. Screenshot of Feature 2 language selection

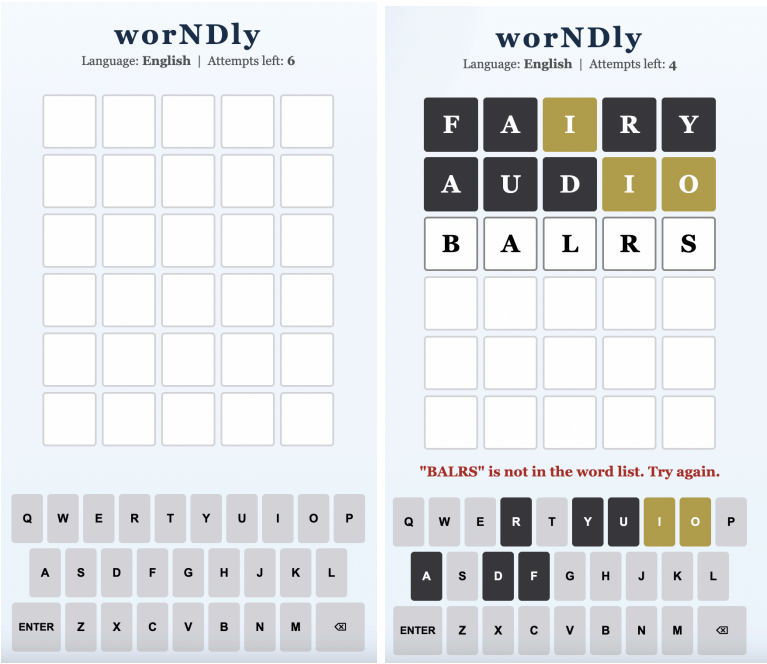


Figure 6. Screenshot of Feature 2 empty grid, word checking, and invalid word



Figure 7. Screenshot of Feature 2 correctly guessed word

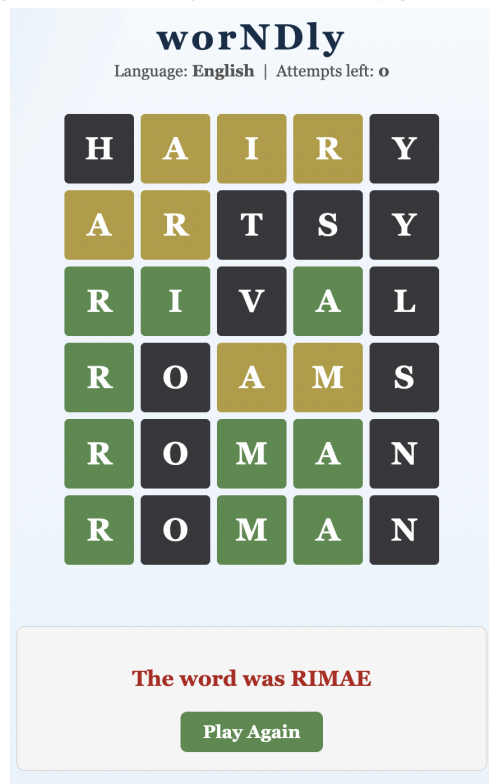


Figure 8. Screenshot of Feature 2 incorrectly guessed word

Feature 3: Player’s Dashboard

Feature 3.1: View All Prior Plays

Subfeature 3.1 allows a logged-in user to review their full game history on the player's dashboard. After signing in, the user navigates to the Dashboard page from the navigation bar. The dashboard queries the database for all game records associated with the authenticated user and displays them in a "Play History" table that lists each game's Date, Language, Result (Pass/Fail), and the Number of Attempts taken.

To help the user explore their history at different time scales, the table includes four filter buttons: This week, This month, This year, and All time. Selecting a filter re-queries the play history and updates the table to show only games played within the selected time range, with the active filter highlighted. *Figure 9* shows the dashboard with the All time filter active, displaying every game the user has played, while *Figure 10* shows the same table after selecting This month, which restricts the results to games played in the current month.

If a user who is not logged in attempts to access the dashboard URL, the system redirects them to the login page so that play history remains private to its owner.

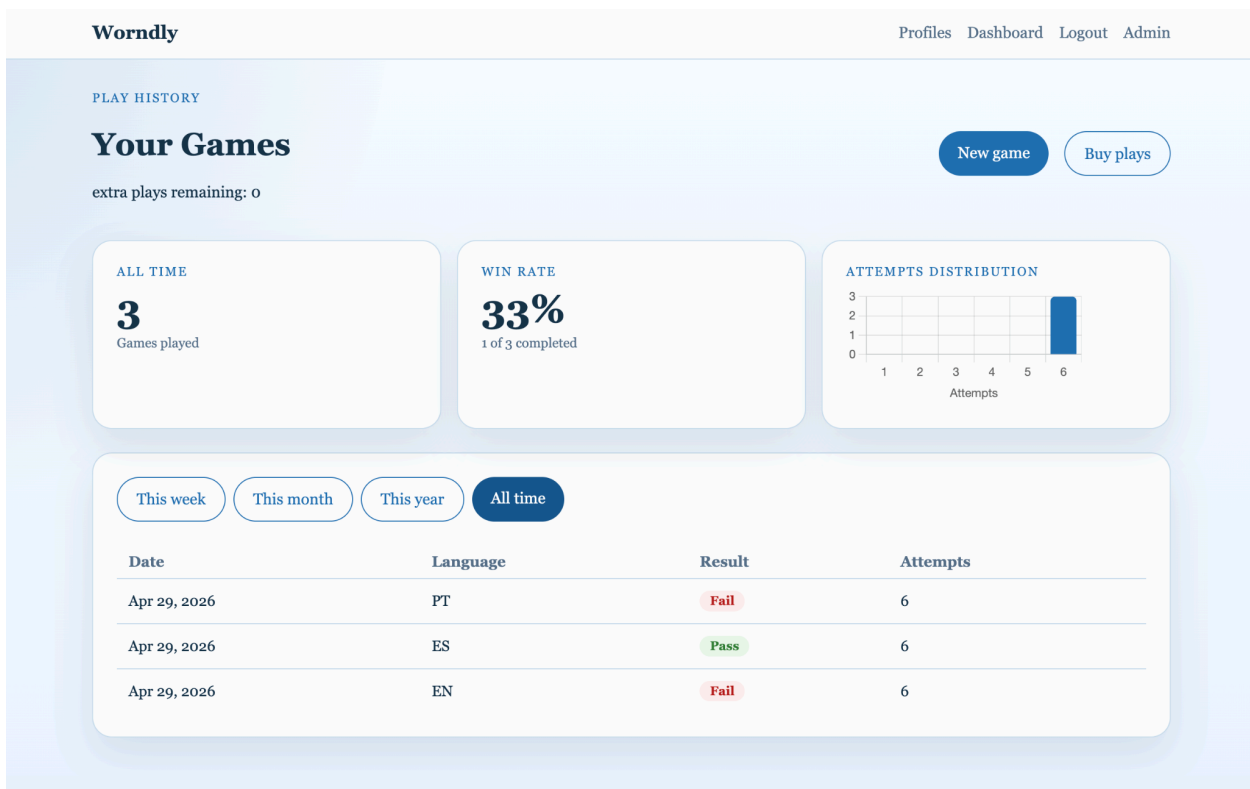


Figure 9. Dashboard for Feature 3

Date	Language	Result	Attempts
Apr 29, 2026	PT	Fail	6
Apr 29, 2026	ES	Pass	6
Apr 29, 2026	EN	Fail	6

Figure 10. “This month” selected for Feature 3

Figure 3.2: Viewing Basic Statistics

Subfeature 3.2 displays a summary of the user's overall performance at the top of the dashboard, computed from the same game history records used in 3.1. Three statistic cards are shown side-by-side:

- All Time: the total number of games the user has played (i.e. “3 Games played” in *Figure 9*).
- Win Rate: the percentage of games the user has won, along with the underlying ratio of wins to total games played (i.e. 33%, 1 of 3 completed”).
- Attempts Distribution: a bar chart, rendered with Chart.js, showing how many of the user's games ended on each possible attempt number from 1 to 6. This gives the player a quick visual sense of whether they tend to solve the puzzle early or run down to their last guesses.

The statistics are computed on the server side from the user's game records each time the dashboard is loaded, so the numbers always reflect the most recent gameplay. *Figure 9* shows the full dashboard with both the statistic cards and the play history table.

Feature 4: Buying Game Plays

Subfeature 4.1 allows a logged-in user to purchase additional game plays using Krato\$Coin through an external REST API. On the purchase page, the user is shown their current Krato\$Coin balance retrieved from the external service and can select how many additional plays they would like to buy. When the form is submitted, the system sends a request to the external payment API using the user’s email address and determines whether the transaction succeeds or fails based on the API response.

If the transaction is successful, the system displays a confirmation message and updates the user’s account by adding the purchased plays to their paid plays remaining. These purchased plays accumulate over time and do not expire after 24 hours. If the transaction fails, the system displays the error message returned by the API and does not modify the user’s remaining plays.

Once a user has exhausted their 3 free daily plays, the application allows continued gameplay only if the user has purchased plays remaining. This feature integrates Worndly with an external payment service while maintaining local control over gameplay access within the application.

Setup / Configuration Note

Feature 4.1 requires a local .env file in the project root to authenticate with the external Krato\$Coin REST API. The .env file should include:

```
KRATOS_ACCESS_TOKEN=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1bl90eXBlljoiYWNjZXNzIiwiaXhwIjozMDc0Nzg4LCJpYXQiOjE3Nzc0MzQ3ODgsImp0aSI6IjRmMzYzYzU1NzJhMTRmY2E4Y2E1NjQxM2U4ZWJmNTlkIiwidXNlcl9pZCI6NDh9.L2_rRRlkudek_YwEykou4YCIdTBL1WtblxNxJO3RGU4
KRATOS_GROUP_PATH=group20/group20
```

You can also use the following account information already set up with Krato\$Coin or use the email when setting up an account:

Username: Matthew

Password: Matt0601\$

Email: mlee55@nd.edu

The screenshot shows the 'Worndly' application interface. At the top, there's a navigation bar with 'Worndly' on the left and 'Profiles', 'Dashboard', 'Logout', and 'Admin' on the right. Below the navigation bar, the page title is 'FEATURE 4.1 Buy extra game plays'. A 'Back to dashboard' button is in the top right. The main content area shows 'paid plays remaining: 10'. Below this, there's a box containing 'current balance: 90 krato\$coin' and 'games to buy:'. There's an input field for 'games to buy' with a dropdown arrow on the right. At the bottom of the box is a 'Buy plays' button.

The screenshot shows the 'Worndly' application interface after a successful purchase. The navigation bar is the same. The page title is 'FEATURE 4.1 Buy extra game plays'. The 'Back to dashboard' button is in the top right. The main content area shows 'paid plays remaining: 15'. Below this, there's a box containing 'current balance: 85 krato\$coin' and 'purchase successful and 5 extra plays were added'. Below the success message is 'games to buy:'. There's an input field for 'games to buy' with a dropdown arrow on the right. At the bottom of the box is a 'Buy plays' button.

Figure 11. Screenshot for Feature 4 showing screen to buy extra game plays and a successful purchase of 5 plays

Project's Learned Lesson

Programming Paradigm

We used a Django MVC/MVT pattern with a database, SQLite. Django provided built-in support for user authentication, database management through ORM, and a structured way to organize our apps logic. These features let us progress quickly with the user accounts and session management without having to build from scratch. If we were to start over, we would use Django again as it provided a good base for our project and good separation/organization.

Most Intellectually Challenging Aspects

The most challenging part of this project was implementing the external Kratos\$Coin REST API for purchasing extra game plays. Unlike our internal database operations, we had no control over the external service, so we had to handle the authentication tokens carefully, parse the varying API responses, and display error messages when the transactions failed. Debugging errors that depending on an external service added a layer of complexity we did not think of beforehand.

A close second was implementing the core Wordle (WorNDly) gameplay logic. While the concepts are simple on the surface, keeping track of the game state between guesses was tricky – ensuring the correct/incorrect guesses displayed in the correct row required careful management of the grid's state. Small bugs would cause letters to appear in the wrong position, overwrite previous guesses, or simply disappear after a next guess was input. Getting the visual feedback and input logic to work together seamlessly took significantly more iteration than we expected.

Positive Process and Organization Process

The most impactful aspect of our project was defining the database and using Django. Having a shared structure (auth_user, player_extension, game, and guess) helped each group participant have a common foundation to build on independently. This prevented having to overexplain and misunderstandings about how data was stored and related.

Additionally, the project's breakdown into clearly defined features and subfeatures made it easy to divide the work and track progress. Each feature had a clear goal which gave us a good understanding of our objective. Overall, maintaining a clear communication about what each

component was responsible for, had the largest positive impact on delivering the project properly.